

Assignment 3 - Bridge Pattern

Topic A - Drawing | Individual | Java, JDK 17

Name: [Your Name] Group: [Your Group]

1. Design and Bridge rationale

The solution separates two independent dimensions. The abstraction dimension is the shape type: **Shape** with **Circle** and **Square**. The implementation dimension is the rendering technology: **Renderer** with **VectorRenderer**, **RasterRenderer**, and the added **AsciiRenderer**. **Shape** stores a **Renderer** reference supplied through its constructor, and the refined abstractions delegate rendering through that interface. This is a direct Bridge structure because the shape hierarchy and renderer hierarchy can vary independently, including runtime replacement of the implementation reference. [1]

Role map

Role	Class	Source
Abstraction	Shape	src/Shape.java
Refined A1	Circle	src/Circle.java
Refined A2	Square	src/Square.java
Implementor	Renderer	src/Renderer.java
I1	VectorRenderer	src/VectorRenderer.java
I2	RasterRenderer	src/RasterRenderer.java
I3	AsciiRenderer	src/AsciiRenderer.java
Client	Main	src/Main.java

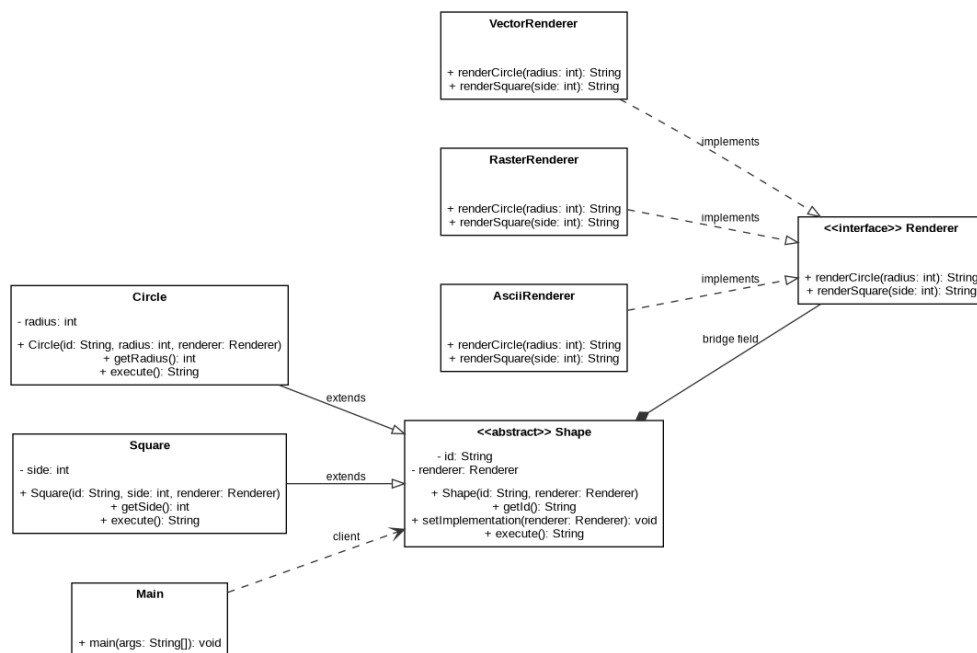


Figure 1. Final UML class diagram. The interface-typed `Renderer` reference in `Shape` is the Bridge connection.

2. Five Clean Code excerpts

Separation of responsibilities

```
public abstract class Shape {
    private Renderer renderer;
    public abstract String execute();
}

public class VectorRenderer implements Renderer {
    public String renderCircle(int radius) {
        return "VECTOR circle radius=" + radius;
    }
}
```

Benefit: Shape handles abstraction state and delegation; the renderer handles low-level rendering output. The responsibilities stay separate.

Meaningful names

```
public void setImplementation(Renderer renderer) {
    this.renderer = renderer;
}

public String renderSquare(int side) {
    return "RASTER square side=" + side;
}
```

Benefit: setImplementation, renderSquare, renderer, radius, and side make the intent and data clear without vague names.

Small focused methods

```
private static boolean check(String id, String classes, Shape shape, String expected) {
    String actual = shape.execute();
    boolean pass = actual.equals(expected);
    System.out.print(id + " " + (pass ? "PASS" : "FAIL")
        + " | " + classes + " | result=" + actual);
    if (!pass) {
        System.out.print(" | expected=" + expected);
    }
    System.out.println();
    return pass;
}
```

Benefit: check performs one focused job: run one real case, compare its result, print evidence, and return the result.

Avoidance of duplication

```
public interface Renderer {
    String renderCircle(int radius);
    String renderSquare(int side);
}

public String execute() {
    return getRenderer().renderCircle(radius);
}
```

Benefit: The shared Renderer contract and delegation avoid creating separate classes for every Circle/Square plus renderer combination.

Encapsulation of state

```
private final String id;
private final int radius;
private Renderer renderer;

public void setImplementation(Renderer renderer) {
    this.renderer = renderer;
}
```

Benefit: ID and domain size are protected from accidental mutation; only the implementation reference is changed through an explicit method.

3. Extension and Bridge versus Adapter

The two-by-two base solution was committed at [2ae0e3cddcc27752c3ce668513a16bfaca685894](#). The extension commit is [39b1f33b5b7bfdd7b96a5e81cfb81afa35d11d8f](#). Only the new AsciiRenderer source and Main changes are present in the src-side extension, with sources.txt updated as allowed by the brief. The existing Shape, Circle, Square, Renderer, VectorRenderer, and RasterRenderer files are unchanged. The resulting extension.diff is the recorded source diff from the base commit to the final commit. [1]

Bridge and Adapter both use composition, but their intent is different. Bridge separates two dimensions that are expected to vary independently; here those dimensions are shape and rendering technology. Adapter is used when an existing class has an incompatible interface and a middle layer translates it to the interface the client expects. This project does not translate a legacy or third-party interface, so Bridge is the appropriate pattern. [1][2]

Trade-off: the pattern adds an interface and more classes for a small application, so the code is more verbose. In exchange, a new renderer can be added without creating new shape-renderer combination subclasses.

4. Demonstration evidence

The fixed samples are Circle radius 2 and Square side 3. The demo uses the required no-input command and compares each real result with an expected result, printing PASS or FAIL from the comparison. The final source compiles with the assignment command using JDK 17 language/API level and no additional dependencies. [3]

```
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | sameObject=true | stateUnchanged=true | before=VECTOR circle radius=2 | after=RASTER circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3
SUMMARY: 7/7 PASS
```

T5 also proves object identity using reference equality (`==`). The same Circle object reference is reused after `setImplementation(...)`; the ID and radius remain unchanged, while the actual rendering description changes from Vector to Raster. This demonstrates runtime replacement of the implementation without replacing the abstraction object or its domain data.

5. Sources

[1] Refactoring Guru. "Bridge." <https://refactoring.guru/design-patterns/bridge>

[2] Refactoring Guru. "Adapter." <https://refactoring.guru/design-patterns/adapter>

[3] Oracle. Java SE 17 & JDK 17 Documentation. <https://docs.oracle.com/en/java/javase/17/>

[4] Astana IT University. Assignment 3 | Bridge Pattern, School of Software Engineering, 2026-2027.